



Session 14

By: Prof. Raksha Sharma

Module 2, Week 6
June 7th 2026



Combining DataFrames and Structural Joins

(Diving deeper in data analysis using Pandas)

Why combining data



- In real scenarios, data is stored in multiple tables
 - Year wise percentage of students in a degree program.
 - Functions to perform Aggregation: Sum, Average (Mean), Count, Minimum, Maximum
 - Employee's personal details in Employee table, with department code their salary in another table with employee IDs, department's details of a company in another table with department IDs.
- **To answer meaningful questions, we often need to combine these tables.**
'What is the salary and department of employee whose name is Harshit Sharma'

Why combining data? Why not everything in one table



- For small organizations with handful of data, it is OK
- For big organizations with huge data, it's not OK
 - Messy data
 - If one column gets corrupted, entire data is affected
 - Addition of an employee, should affect only employee table and salary table, why department table?

High level questions from multiple tables



Tables:

- Employees
- Departments
- Salaries

Questions:

- *What is the average salary by department?*
- *Which department has the highest payroll?*
- *How many employees work in each department?*

First thing to do



1. Import Pandas -> create/take tables in a data-frame
 - You have already created dataset in tabular format using pandas
2. Get to know the functions of Pandas allowing merging of two or more than two tables

pd.concat() *Stack similar tables together*

pd.merge() *Join related tables using a common key*

<https://colab.research.google.com/drive/1puNyp9zM7AEo6jMMPdJocEFE6F6XlMm5?usp=sharing>

Concatanation of Two tables



- Concatanation is possible for two similar tables, having similar attributes.
- Horizontal and Vertical (default increasing row dimensions)

Jan and feb are two dataframes having sales record of january and february

```
sales = pd.concat([jan, feb], ignore_index=True)
```

#ignore_index=True creates a fresh index

Join (Merging) of Tables using Keys



- **Keys:** Datasets are connected through common columns called keys.

Employee Table (emp_id, name, department_code) ~ Salary Table(emp_id, salary)

- *Emp_id is key (creating bridge between two tables to combine them)*

Dataset 1

Employees
Students
Customers
Products

Common Key

Employee_ID
Student_ID
Customer_ID
Product_ID

Dataset 2

Salaries
Marks
Orders
Sales

Join (Merging) of Tables using Keys: Types of Join

- Inner Join: Returns only matching records from both tables.

```
pd.merge(employees,salary, on="Employee_ID", how="inner")
```

- Left Join: Returns matching and remaining rows of the left table

```
pd.merge(employees,salary, on="Employee_ID", how="left")
```

- Right Join: Returns matching and remaining rows of the right table

```
pd.merge(employees,salary, on="Employee_ID", how="right")
```

- Outer Join: Keeps all rows from both tables.

```
pd.merge(employees,salary, on="Employee_ID", how="outer")
```

Join (Merging) of Tables using Keys: Types of Join

Join Type	Meaning	When to Use
Inner Join	Keeps only matching records	When only complete matched data is needed
Left Join	Keeps all records from left table	When left table is the main table
Right Join	Keeps all records from right table	When right table is the main table
Outer Join and gaps	Keeps all records from both tables	When you want to find all matches



Thank you!